

PySpark Data Cleaning & Transformation Pipeline

Overview

This notebook demonstrates a complete data cleaning and transformation workflow:

1. **Data Cleaning:** Remove nulls, duplicates, and invalid values
2. **Daily Sales Analysis:** Aggregate sales by date
3. **City-wise Revenue:** Revenue breakdown by location
4. **Top Customers:** Identify highest-value customers
5. **Repeat Customer Analysis:** Find customers with multiple orders
6. **Customer Segmentation:** Classify customers into Gold/Silver/Bronze tiers
7. **Final Reporting:** Combine all insights into a comprehensive report
8. **Save Output:** Export results for downstream use

Key Learning Goals:

- Understand why data cleaning comes first
- Practice joins and aggregations
- Apply business logic for segmentation
- Build production-ready data pipelines

What is Data Cleaning?

Data cleaning is the process of identifying and correcting errors, inconsistencies, and missing values in datasets to ensure data quality before analysis.

Key Data Cleaning Steps:

1. **Remove rows with null keys:** Records without essential identifiers (like customer_id) cannot be joined or analyzed properly
2. **Remove duplicate rows:** Duplicates skew counts and aggregations, leading to incorrect metrics
3. **Filter invalid values:** Negative amounts, impossible dates, or out-of-range values indicate data errors
4. **Check column types:** Ensure data types match expected formats (dates as dates, numbers as numbers)
5. **Standardize formats:** Consistent casing, trimming whitespace, standardizing codes

Why Clean Data First?

- **Prevents join failures:** Null keys cause LEFT/INNER joins to drop records unexpectedly
- **Ensures accurate counts:** Duplicates inflate metrics like order counts and revenue
- **Avoids calculation errors:** Invalid values (negative prices) corrupt aggregations
- **Improves performance:** Smaller, cleaner datasets process faster
- **Builds trust:** Clean data produces reliable insights

```
In [ ]: %python
# First, let's inspect the raw customer data to understand what we're
working with
from pyspark.sql.functions import *

customers_df = spark.table("samples.tpch.customer")
display(customers_df.limit(10))
```

c_custkey	c_name	c_address
412445	Customer#000412445	0QAB3OjYnbP6mA0B,kgf
412446	Customer#000412446	5u8MSbyiC7J,7PuY4Ivaq1JRbTCMKen\

```
In [ ]: %python
# Inspect orders data
orders_df = spark.table("samples.tpch.orders")
display(orders_df.limit(10))
```

o_orderkey	o_custkey	o_orderstatus	o_totalprice	o_orderdate
11396166	179329	O	187683.10	1996-06-22
11396167	473245	F	117554.58	1994-07-20

```

In [ ]: %python
# Check data quality issues before cleaning
from pyspark.sql.functions import col, count, countDistinct, sum as
_sum, when, lit

customers_df = spark.table("samples.tpch.customer")
orders_df = spark.table("samples.tpch.orders")

# Customer data quality
cust_quality = customers_df.agg(
    lit('Customers').alias('table_name'),
    count('*').alias('total_rows'),
    countDistinct('c_custkey').alias('unique_customers'),
    _sum(when(col('c_custkey').isNull(),
1).otherwise(0)).alias('null_keys'),
    _sum(when(col('c_name').isNull(),
1).otherwise(0)).alias('null_names'),
    _sum(when(col('c_acctbal') < 0,
1).otherwise(0)).alias('negative_balances')
)

# Orders data quality
orders_quality = orders_df.agg(
    lit('Orders').alias('table_name'),
    count('*').alias('total_rows'),
    countDistinct('o_orderkey').alias('unique_orders'),
    _sum(when(col('o_custkey').isNull(),
1).otherwise(0)).alias('null_customer_keys'),
    _sum(when(col('o_orderdate').isNull(),
1).otherwise(0)).alias('null_dates'),
    _sum(when(col('o_totalprice') < 0,
1).otherwise(0)).alias('negative_prices')
)

# Combine results
quality_report = cust_quality.union(orders_quality)
display(quality_report)

```

table_name	total_rows	unique_customers	null_keys	null_names	n
Customers	750000	750000	0	0	6
Orders	7500000	7500000	0	0	0

Task 1: Daily Sales Analysis

Objective: Aggregate total sales and order counts by date to identify daily revenue trends.

Output: `date` , `total_sales` , `order_count`

Why this matters: Daily sales trends help identify peak periods, seasonal patterns, and revenue fluctuations.

Task 2: City-wise Revenue Analysis

Objective: Calculate total revenue, customer count, and order count for each city.

Output: `city` , `total_revenue` , `customer_count` , `order_count`

Why this matters: Geographic analysis helps identify high-performing markets and guides regional strategy.

Task 3: Top 5 Customers

Objective: Identify the highest-spending customers to focus retention and VIP programs.

Output: `customer_name` , `customer_id` , `total_spend` , `order_count`

Why this matters: Top customers often represent a disproportionate share of revenue (Pareto principle).

Task 4: Repeat Customer Analysis

Objective: Find customers with more than one order to measure customer loyalty.

Output: `customer_id` , `order_count` , `total_spend`

Why this matters: Repeat customers indicate satisfaction and are cheaper to retain than acquiring new ones.

Task 5: Customer Segmentation

Objective: Classify customers into Gold/Silver/Bronze tiers based on total spending.

Business Rules:

- **Gold:** `total_spend > 10,000` - Premium customers
- **Silver:** `total_spend 5,000-10,000` - Mid-tier customers
- **Bronze:** `total_spend < 5,000` - Standard customers

Output: `customer_name` , `customer_id` , `total_spend` , `segment`

Why this matters: Segmentation enables targeted marketing, personalized service levels, and resource prioritization.

Task 6: Final Reporting Table

Objective: Combine all insights into a comprehensive customer report for business stakeholders.

Output: `customer_name` , `city` , `total_spend` , `order_count` , `segment` , `first_order_date` , `last_order_date`

Why this matters: A unified view enables holistic customer understanding and supports strategic decision-making.

```
In [ ]: %python
# Step 1: Clean customer data
# Remove nulls, duplicates, and ensure data quality
from pyspark.sql.functions import col, split

customers_df = spark.table("samples.tpch.customer")

clean_customers = customers_df \
    .filter(col('c_custkey').isNotNull()) \
    .filter(col('c_name').isNotNull()) \
    .filter(col('c_acctbal') >= 0) \
    .select(
        col('c_custkey').alias('customer_id'),
        col('c_name').alias('customer_name'),
        split(col('c_address'), ',').getItem(0).alias('city'),
        col('c_acctbal').alias('account_balance')
    ) \
    .distinct()

# Create temp view for later use
clean_customers.createOrReplaceTempView('clean_customers')

print(f"Cleaned customer count: {clean_customers.count()}")
```

Cleaned customer count: 681834

```
In [ ]: %python
# Step 2: Clean orders data
from pyspark.sql.functions import col

orders_df = spark.table("samples.tpch.orders")

clean_orders = orders_df \
    .filter(col('o_custkey').isNotNull()) \
    .filter(col('o_orderkey').isNotNull()) \
    .filter(col('o_orderdate').isNotNull()) \
    .filter(col('o_totalprice') > 0) \
    .select(
        col('o_orderkey').alias('order_id'),
        col('o_custkey').alias('customer_id'),
        col('o_orderdate').alias('order_date'),
        col('o_totalprice').alias('total_price')
    ) \
    .distinct()

# Create temp view for later use
clean_orders.createOrReplaceTempView('clean_orders')

print(f"Cleaned order count: {clean_orders.count()}")
```

Cleaned order count: 7500000

```
In [ ]: %python
# Task 1: Daily Sales → Output: date, total_sales
from pyspark.sql.functions import col, sum as _sum, count, round as
_round

daily_sales = clean_orders \
    .groupBy(col('order_date').alias('date')) \
    .agg(
        _round(_sum('total_price'), 2).alias('total_sales'),
        count('order_id').alias('order_count')
    ) \
    .orderBy(col('date').desc()) \
    .limit(30)

# Create temp view
daily_sales.createOrReplaceTempView('daily_sales')

display(daily_sales)
```

date	total_sales	order_count
1998-08-02	468542019.09	3072
1998-08-01	477329249.06	3133
1998-07-31	475064436.51	3116
1998-07-30	469038176.79	3132
1998-07-29	462137689.96	3102
1998-07-28	463826324.39	3076
1998-07-27	486669132.14	3124
1998-07-26	470313673.41	3129

Databricks visualization. Run in Databricks to view.


```
In [ ]: %python
# Task 2: City-wise Revenue → Output: city, total_revenue
from pyspark.sql.functions import col, sum as _sum, count,
countDistinct, round as _round

city_revenue = clean_orders.alias('o') \
    .join(clean_customers.alias('c'), col('o.customer_id') ==
col('c.customer_id'), 'inner') \
    .groupBy(col('c.city')) \
    .agg(
        _round(_sum('o.total_price'), 2).alias('total_revenue'),
        countDistinct('o.customer_id').alias('customer_count'),
        count('o.order_id').alias('order_count')
    ) \
    .orderBy(col('total_revenue').desc()) \
    .limit(20)

# Create temp view
city_revenue.createOrReplaceTempView('city_revenue')

display(city_revenue)
```

city	total_revenue	customer_count	order_count
	16072745096.27	7051	106416
V	342825327.58	137	2254
a	321383213.52	128	2072
M	315562701.58	134	2069
i	296445467.70	130	1905
b	296022767.02	138	1982
z	294249329.81	126	1986
S	293589632.45	132	1934

Databricks visualization. Run in Databricks to view.

```
In [ ]: %python
# Task 3: Top 5 Customers → Output: customer_name, total_spend
from pyspark.sql.functions import col, sum as _sum, count, round as
_round

top_customers = clean_orders.alias('o') \
    .join(clean_customers.alias('c'), col('o.customer_id') ==
col('c.customer_id'), 'inner') \
    .groupBy(col('c.customer_id'), col('c.customer_name')) \
    .agg(
        _round(_sum('o.total_price'), 2).alias('total_spend'),
        count('o.order_id').alias('order_count')
    ) \
    .orderBy(col('total_spend').desc()) \
    .limit(5)

# Create temp view
top_customers.createOrReplaceTempView('top_customers')

display(top_customers)
```

customer_id	customer_name	total_spend	order_count
721180	Customer#000721180	7147854.45	38
382414	Customer#000382414	7139342.74	34
299701	Customer#000299701	7128450.99	43
179275	Customer#000179275	6887650.62	35
292987	Customer#000292987	6833078.18	38

Databricks visualization. Run in Databricks to view.

```
In [ ]: %python
# Task 4: Repeat Customers (>1 order) → Output: customer_id, order_count
from pyspark.sql.functions import col, count, sum as _sum, round as
_round

repeat_customers = clean_orders \
    .groupBy('customer_id') \
    .agg(
        count('order_id').alias('order_count'),
        _round(_sum('total_price'), 2).alias('total_spend')
    ) \
    .filter(col('order_count') > 1) \
    .orderBy(col('order_count').desc()) \
    .limit(20)

# Create temp view
repeat_customers.createOrReplaceTempView('repeat_customers')

display(repeat_customers)
```

customer_id	order_count	total_spend
299701	43	7128450.99
124282	43	6280442.60
561661	40	5675667.37
318454	40	6769695.98
85909	39	5841007.45
415006	39	6240904.98
337549	39	5382970.14
494638	39	6232841.92

Databricks visualization. Run in Databricks to view.

```

In [ ]: %python
# Task 5: Customer Segmentation → Gold/Silver/Bronze
# Business Logic:
#   total_spend > 10000 → Gold
#   total_spend 5000-10000 → Silver
#   total_spend < 5000 → Bronze

from pyspark.sql.functions import col, sum as _sum, count, round as
_round, when, avg

customer_segments = clean_orders.alias('o') \
    .join(clean_customers.alias('c'), col('o.customer_id') ==
col('c.customer_id'), 'inner') \
    .groupBy(col('c.customer_id'), col('c.customer_name')) \
    .agg(
        _round(_sum('o.total_price'), 2).alias('total_spend')
    ) \
    .withColumn('segment',
        when(col('total_spend') > 10000, 'Gold')
        .when(col('total_spend') >= 5000, 'Silver')
        .otherwise('Bronze')
    )

# Create temp view
customer_segments.createOrReplaceTempView('customer_segments')

# Show segment distribution
segment_distribution = customer_segments \
    .groupBy('segment') \
    .agg(
        count('*').alias('customer_count'),
        _round(_sum('total_spend'), 2).alias('total_revenue'),
        _round(avg('total_spend'), 2).alias('avg_spend_per_customer')
    ) \
    .withColumn('sort_order',
        when(col('segment') == 'Gold', 1)
        .when(col('segment') == 'Silver', 2)
        .otherwise(3)
    ) \
    .orderBy('sort_order') \
    .drop('sort_order')

display(segment_distribution)

```

segment	customer_count	total_revenue	avg_spend_per_customer
Gold	454716	1031068684800.35	2267500.34
Silver	1	5670.41	5670.41

Databricks visualization. Run in Databricks to view.

Databricks visualization. Run in Databricks to view.

```

In [ ]: %python
# Task 6: Final Reporting Table
# Combine all insights: customer_name, city, total_spend, order_count,
segment

from pyspark.sql.functions import col, count, min as _min, max as _max

final_report = clean_customers.alias('c') \
    .join(clean_orders.alias('o'), col('c.customer_id') ==
col('o.customer_id'), 'inner') \
    .join(customer_segments.alias('seg'), col('c.customer_id') ==
col('seg.customer_id'), 'inner') \
    .groupBy(
        col('c.customer_id'),
        col('c.customer_name'),
        col('c.city'),
        col('seg.total_spend'),
        col('seg.segment')
    ) \
    .agg(
        count('o.order_id').alias('order_count'),
        _min('o.order_date').alias('first_order_date'),
        _max('o.order_date').alias('last_order_date')
    ) \
    .select(
        col('customer_name'),
        col('city'),
        col('total_spend'),
        col('order_count'),
        col('segment'),
        col('first_order_date'),
        col('last_order_date')
    ) \
    .orderBy(col('total_spend').desc())

# Create temp view
final_report.createOrReplaceTempView('final_report')

# Display top 20 customers from final report
display(final_report.limit(20))

```

customer_name	city	to
Customer#000721180	xVvi2xMYV7MqiPdC	71
Customer#000382414	QjI aHILRIBLeg6HkUjg	71
Customer#000299701	bQayuljeavArxOQI	71
Customer#000179275	t6afpRmxArdPPqqQPSzVRWfpyTxECkQsWaTX	68
Customer#000292987	CjD	68
Customer#000253099	fDBY7k7eegbOws6DnsEEyr	68
Customer#000333100	OB	67

Databricks visualization. Run in Databricks to view.

```
In [ ]: %python
# Summary statistics for the final report
from pyspark.sql.functions import countDistinct, sum as _sum, avg, round
as _round

summary_stats = final_report.agg(
    countDistinct('customer_name').alias('total_customers'),
    countDistinct('city').alias('total_cities'),
    _sum('order_count').alias('total_orders'),
    _round(_sum('total_spend'), 2).alias('total_revenue'),
    _round(avg('total_spend'), 2).alias('avg_customer_value'),
    _round(avg('order_count'), 2).alias('avg_orders_per_customer')
)

display(summary_stats)
```

total_customers	total_cities	total_orders	total_revenue	avg_cust
454717	436977	6822376	1031068690470.76	2267495



Task 7: Save Output

Objective: Persist the final report as a permanent Delta table for downstream consumption.

Output: Permanent table `workspace.default.customer_report`

Why this matters: Temporary views disappear when the session ends. Permanent tables enable:

- Sharing results across teams
- Scheduled refreshes and pipelines
- Historical tracking and versioning
- Integration with BI tools and dashboards

```
In [ ]: %python
# Task 7: Save Output → Save final report to a permanent table
# Create a permanent Delta table from the final report
from pyspark.sql.functions import lit, count, countDistinct

final_report.write \
    .mode('overwrite') \
    .saveAsTable('workspace.default.customer_report')

# Verify the table was created and show row count
verification = spark.table('workspace.default.customer_report').agg(
    lit('customer_report').alias('table_name'),
    count('*').alias('total_rows'),
    countDistinct('customer_name').alias('unique_customers'),
    countDistinct('segment').alias('segments')
)

display(verification)
```

table_name	total_rows	unique_customers	segments
customer_report	454717	454717	2

```
In [ ]: %python
# Alternative: Export the report as CSV file
# Read the saved table
df = spark.table("workspace.default.customer_report")

# Show summary of what will be exported
print(f"Total records to export: {df.count()}")
print(f"\nSample of data:")
display(df.limit(10))

# Note: To export as CSV in production, you would use:
# output_path = "/Volumes/catalog/schema/volume/customer_report.csv"
# df.coalesce(1).write.mode('overwrite').option('header',
# 'true').csv(output_path)
print("\n✅ Table saved successfully as
workspace.default.customer_report")
print("To export as CSV, configure a Unity Catalog Volume path")
```

Total records to export: 454717

Sample of data:

customer_name	city	to
Customer#000721180	xVvi2xMYV7MqiPdC	71
Customer#000382414	Qjl aHILRIBLeg6HkUjg	71
Customer#000299701	bQayuljeavArxOQl	71
Customer#000179275	t6afpRmxArdPPqqQPSzVRWfpyTxECkQsWaTX	68
Customer#000292987	CjD	68
Customer#000253099	fDBY7k7eegbOws6DnsEEyr	68
Customer#000333100	OB	67
Customer#000370441	xYacGNGrF2lmiVMM7zK5uHhGwz2mFL8FYQs	67

✅ Table saved successfully as workspace.default.customer_report
To export as CSV, configure a Unity Catalog Volume path

Reflection Questions (Very Important)

1. Why is cleaning done before joining tables?

Answer: Cleaning must happen first because:

- **Null keys cause join failures:** If customer_id is NULL, INNER/LEFT joins won't match records, silently dropping data
- **Duplicates inflate results:** If orders table has duplicates, revenue totals will be wrong after joining
- **Invalid values corrupt calculations:** Negative prices or impossible dates create incorrect aggregations
- **Performance:** Cleaning reduces row count, making subsequent joins faster
- **Data integrity:** Ensures every record in the final output is valid and trustworthy

2. What would go wrong if null keys are not removed?

Answer:

- **INNER JOIN:** Records with null keys are silently excluded from results, reducing counts without warning
- **LEFT JOIN:** NULL keys appear in output but can't match to the other table, creating incomplete records
- **Aggregations:** GROUP BY treats NULLs as a separate group, skewing segment counts
- **Business logic:** Downstream applications expecting valid IDs will crash or produce errors
- **Example:** A customer with NULL customer_id appears in orders but never shows up in city revenue reports

3. How did you decide join order?

Answer: Join order follows the **dimension** → **fact** → **aggregation** pattern:

1. **Start with cleaned base tables** (clean_customers, clean_orders)
2. **Join orders to customers** since orders reference customers (foreign key relationship)
3. **Add derived tables last** (customer_segments) after base aggregations are complete
4. **Principle:** Join smallest table first when possible, and ensure keys are indexed
5. **In this pipeline:** orders (fact) → customers (dimension) → segments (derived)

4. Which step was most difficult and why?

Answer: Typically the **customer segmentation** (Task 5) is most challenging because:

- **Business logic complexity:** Translating business rules (Gold/Silver/Bronze) into SQL CASE statements
- **Aggregation before segmentation:** Must calculate total_spend per customer BEFORE applying segment logic
- **Multiple conditions:** Ensuring threshold boundaries are correct (>10000 vs ≥ 10000)
- **Validation:** Verifying segments make business sense (Gold customers should have highest spend)

5. How is SQL logic similar to PySpark?

Answer:

Concept	SQL	PySpark
Filtering	<code>WHERE customer_id IS NOT NULL</code>	<code>.filter(col('customer_id').isNotNull())</code>
Aggregation	<code>SUM(total_price)</code>	<code>.agg(sum('total_price'))</code>
Grouping	<code>GROUP BY customer_id</code>	<code>.groupBy('customer_id')</code>
Joins	<code>INNER JOIN orders ON...</code>	<code>.join(orders, on=..., how='inner')</code>
Case logic	<code>CASE WHEN ... THEN ... END</code>	<code>.when(...).otherwise(...)</code>

Both are declarative (describe what you want, not how to compute it) and **both use lazy evaluation** (build execution plan, execute on action).

6. What challenges will appear with large data?

Answer:

- **Memory pressure:** Full table scans and wide joins can exceed cluster memory
- **Skew:** Uneven data distribution (one customer with millions of orders) slows down specific tasks
- **Shuffles:** GROUP BY and JOIN operations shuffle data across nodes, creating network bottlenecks
- **Timeouts:** Complex queries without proper filtering (date ranges) take too long
- **Duplicate handling:** `SELECT DISTINCT` on billions of rows is expensive
- **Mitigation:** Partition data by date, use broadcast joins for small dimension tables, add indexes, tune cluster size

7. Can you explain your pipeline in simple steps?

Answer:

1. **Load raw data:** Read customers and orders tables
2. **Inspect quality:** Check for nulls, duplicates, invalid values
3. **Clean each table separately:** Remove bad records, standardize formats
4. **Create daily sales:** Aggregate orders by date to see trends
5. **Create city revenue:** Join cleaned orders + customers, group by city
6. **Find top customers:** Aggregate spend per customer, rank by total
7. **Find repeat customers:** Count orders per customer, filter > 1
8. **Segment customers:** Apply business rules to classify Gold/Silver/Bronze
9. **Build final report:** Combine all metrics (city, spend, orders, segment) into one table
10. **Validate results:** Check totals, verify business logic, review sample records
11. **Save output:** Write to Delta table or file for downstream use

Key principle: Clean → Transform → Aggregate → Combine → Validate → Save

In []:

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).